FISIO MARCA / GUÍA RÁPIDA DE DEFENSA

Mi exposición de backend en 10 páginas

Solo cinco módulos: Auth · API Gateway · Carrito · Reservas · Eureka.

[LEER EN VOZ ALTA] · APERTURA (aprox. 1 minuto y 35 segundos)

“Buenos días. Mi parte de FisioMarca se concentra en cinco piezas del backend que trabajan juntas para atender a un paciente de manera organizada y controlada.

Primero, **Auth** verifica quién es el usuario: comprueba su contraseña y le entrega un token JWT, que funciona como una credencial digital.

Segundo, el **API Gateway** es la puerta de entrada: recibe las solicitudes, revisa que el token sea válido cuando corresponde y las dirige al servicio correcto.

Tercero, el **Carrito** permite reunir sesiones de fisioterapia, calcular cantidades y subtotales y validar que el paciente solo manipule su compra.

Cuarto, **Reservas** se encarga de los horarios, la disponibilidad, las citas y el pago de demostración. Evita conflictos de agenda con validaciones y retiene los horarios unos minutos mientras se paga.

Finalmente, **Eureka** es el directorio donde se registran los servicios. Ayuda a encontrarlos sin depender de direcciones fijas.

Lo más importante es que el Gateway valida el token, pero cada microservicio también aplica sus permisos. Así separamos responsabilidades y agregamos controles en más de una capa.”

El mapa mental de mi exposición

Acceso → Identidad → Operación → Cita. Gateway recibe; Auth identifica; Carrito prepara; Reservas agenda; Eureka permite localizar servicios.

Cómo leer: los bloques **[LEER EN VOZ ALTA]** son tu discurso. Los **[Saber por si preguntan]** son respaldo técnico.

01 Flujo completo · de entrar a autenticarse

Cuenta esta parte como una historia, no como una lista de clases.

1. Llegada al sistema El cliente inicia una acción. Una solicitud de API llega a **Gateway :8080**. No todas las operaciones requieren identidad: existen rutas públicas, por ejemplo **POST /auth/register** y **POST /auth/login**.

2. Registro Auth recibe los datos, verifica el correo, crea Cliente y Usuario y guarda una **huella BCrypt** de la contraseña, nunca la contraseña legible. El rol inicial es **CLIENTE**.

3. Login Se envían correo y contraseña a **POST /auth/login**. **AuthenticationManager** consulta al usuario y **BCrypt** compara la contraseña con la huella almacenada. Un error no entrega credencial.

4. JWT firmado Con credenciales correctas, **JwtService** genera el token con correo, **roles**, usuarioId y, si corresponde, clienteId. Firma usando **HS256**. La configuración marca **1 hora** de vigencia.

5. Solicitud protegida El cliente presenta **Authorization: Bearer <token>**. El filtro global del Gateway verifica la **firma** y **expiración**. Sin credencial válida responde **401**.

6. Servicio y permiso El Gateway encuentra el destino mediante **Eureka + lb://NOMBRE-SERVICIO**. El microservicio receptor vuelve a validar JWT y decide **qué acción está permitida** según rol y datos del propietario.

[Saber por si preguntan] · DOS VERIFICACIONES DISTINTAS

El Gateway comprueba que el token sea auténtico y vigente. **Carrito y Reservas** también lo validan y comprueban si el **clienteId** solicitado coincide con el del JWT, salvo para administradores. **Autenticación** no es lo mismo que **autorización**.

02 Flujo completo · del carrito al cierre

Aquí explicas una compra completa sin entrar en módulos ajenos.

7. Armar el carrito Con JWT, el paciente usa **POST /carrito/items** y **GET /carrito/{clienteId}**. Carrito confirma que el cliente exista y obtiene precio/nombre del servicio consultado por **Feign**; suma subtotales. Máximo: **4 sesiones**.

8. Elegir cita y hacer checkout La solicitud **POST /reservas/checkout** incluye el clienteId y una lista de sesiones con fecha y hora. Reservas verifica identidad del dueño, horarios, cupos y límites. Crea una **orden** y deja las citas en **PENDIENTE_PAGO**.

9. Retener horario Reservas aparta el espacio durante **10 minutos**. Si vence, lo cambia a **EXPIRADA**; una tarea programada revisa vencimientos cada **30 segundos**. El paciente tiene un tiempo limitado para completar el pago.

10. Registrar pago demo POST /reservas/orden/{codigo}/pagar valida datos de prueba de tarjeta, Yape o Plin. Si son aceptados, las citas pasan a **PROGRAMADA**. **No existe confirmación bancaria real** en este código.

11. Consultar y cerrar sesión El usuario puede ver su orden y citas, o cancelar según las reglas. Al cerrar sesión, el cliente debe dejar de usar su token. **Auth no tiene endpoint de revocación o logout**: el JWT ya emitido seguirá criptográficamente válido hasta vencer.

[Saber por si preguntan] · NO INVENTES UNA LLAMADA

En los archivos backend revisados, **Carrito no invoca directamente a Reservas**. El endpoint de checkout recibe una lista de sesiones. Explica que Carrito prepara la compra y Reservas procesa la orden; no afirmes que existe una llamada Feign Carrito → Reservas si no aparece en el código.

03 Seguridad · preguntas difíciles

Respuestas muy cortas, pero precisas: aquí puedes ganar puntos.

¿Dónde se guardan los tokens? Auth devuelve el JWT en el JSON del login; el backend no decide dónde persistirlo. El ZIP no trae el código cliente para verificar si usa memoria o cookie. En producción, se debe definir almacenamiento protegido y mitigar XSS/CSRF según el diseño.

¿Qué pasa si interceptan la comunicación? La **firma HS256 evita modificarlo sin la clave**, pero **no oculta** el contenido. Quien robe un Bearer válido puede reutilizarlo hasta que venza. Se necesita **HTTPS/TLS**; el despliegue de demostración usa HTTP.

¿Cómo sabe Gateway que tengo permiso? No comprueba permisos granulares: solo firma, formato y expiración. Luego **Spring Security** y **AccesoCliente** aplican roles y comparan el clienteId del token con el solicitado.

¿Qué diferencias hay entre 401 y 403? 401: falta el token o es inválido/expiró. **403:** el usuario ya está identificado, pero su rol o el clienteId no autorizan esa operación.

¿Si cierran sesión, el token queda inválido? No de forma inmediata. No hay lista negra, revocación ni refresh token en Auth. Expira al cumplirse su tiempo, **3 600 000 ms = 1 hora**, salvo un cambio del secreto de firma.

¿Eureka hace segura la comunicación? No. Eureka descubre direcciones; no cifra las llamadas ni autentica por sí solo a los servicios. Docker deja los microservicios sin puertos públicos, pero para producción hacen falta controles de red, identidad de servicios y TLS.

[Saber por si preguntan] · LÍMITES DE LA DEMO

El YAML y Docker Compose incluyen **secreto JWT de ejemplo** y valores predeterminados de MySQL. Para producción: secretos gestionados fuera del código, HTTPS, control de acceso a Eureka y, si procede, firma asimétrica para no compartir la clave de emisión.

04 Diapositiva 1 · Auth

Analogía: la recepción que revisa identidad y entrega credenciales.

[LEER EN VOZ ALTA] · QUÉ DECIR

"Auth es la recepción de FisioMarca. Cuando alguien se registra, crea su usuario y protege su contraseña con **BCrypt**. Cuando inicia sesión, verifica el correo y la contraseña; si son correctos, entrega un **JWT firmado** con la identidad y el rol. No es una sesión guardada en el servidor: el token viaja en las solicitudes. Los demás módulos pueden verificarlo sin pedir otra vez la contraseña."

Tres ideas que debes dominar

- **Registro: POST /auth/register**; crea CLIENTE y almacena passwordHash.
- **Login: AuthenticationManager** + BCrypt verifican credenciales.
- **Token: JwtService.generateToken()** firma HS256; agrega roles, usuarioId y clienteId.

Tres preguntas probables

¿Se puede recuperar una contraseña con BCrypt? No. Es un hash de una sola dirección; se comparan huellas, no se descifra.

¿Qué contiene el JWT? Correo como subject; roles, usuarioId, nombre, clienteId si existe; emisión y expiración.

¿OAuth2 y JWT son lo mismo? No. JWT es formato de token. Se usa soporte **OAuth2 Resource Server** para validar JWT, sin que eso implique un proveedor OAuth2 externo.

Ubica en el código: AuthController.login() · **UsuarioServices.registrarCliente()** · **JwtService.generateToken()** · **SecurityConfig**.

[Saber por si preguntan]

La clave HS256 se comparte entre el emisor y quienes validan. Si se filtra, un atacante podría firmar tokens falsos. En producción conviene extraerla del código y protegerla.

05 Diapositiva 2 · API Gateway

Analogía: el vigilante y recepcionista de la entrada principal.

[LEER EN VOZ ALTA] · QUÉ DECIR

“El Gateway es el punto de entrada único para las solicitudes de API. Recibe una petición, identifica su ruta y decide a qué microservicio enviarla. Antes, su filtro global revisa el JWT cuando la ruta está protegida. Si falta o venció, devuelve 401; si es correcto, conserva la cabecera Authorization y reenvía la petición. Para encontrar el destino, usa los nombres registrados en Eureka. El permiso específico se verifica dentro de cada microservicio.”

Tres ideas que debes dominar

- **Enrutamiento: /auth/**, /carrito/**, /reservas/**** → sus servicios; puerto **8080**.
- **Filtro: JwtAuthenticationFilter** verifica firma HS256 y expiración.
- **Descubrimiento: lb://MICRO-CARRITO** y similares resuelven instancias por Eureka.

Tres preguntas probables

¿Por qué no basta Gateway para proteger todo? Porque el microservicio conoce el rol necesario y a quién pertenecen los datos.

¿Qué pasa con una ruta pública? El filtro permite login y registro sin Bearer; las rutas protegidas sí lo exigen.

¿Si cambian los puertos de un microservicio? Eureka actualiza la ubicación y Gateway sigue usando su nombre lógico.

Ubica en el código: Gateway/application.yml (routes) · **JwtAuthenticationFilter.filter()** (validación).

[Saber por si preguntan]

El filtro global **no comprueba roles**. Tampoco es TLS: el HTTP de la demo no impide interceptación. CORS define orígenes web permitidos, pero **no sustituye autenticación**.

06 Diapositiva 3 · Carrito

Analogía: una cesta de sesiones antes de escoger horarios.

[LEER EN VOZ ALTA] · QUÉ DECIR

"El microservicio Carrito reúne los tratamientos que una persona quiere adquirir. Permite agregar, actualizar cantidad, quitar y vaciar. No acepta el precio como verdad absoluta: consulta por Feign los datos actuales del servicio y calcula precio unitario y subtotal. La regla principal es un máximo de cuatro sesiones por compra. Además, verifica que un paciente solo pueda trabajar con su propio carrito usando el clienteId firmado en su JWT."

Tres ideas que debes dominar

- **Entidades: Carrito** pertenece a clienteId; **CarritoItem** guarda servicioId, cantidad, precio y subtotal.
- **Reglas: MAX_SESIONES = 4**; suma cantidades, no solo tipos de servicio.
- **Seguridad: AccesoCliente.verificar()** compara el clienteId de URL/DTO con el claim del JWT.

Tres preguntas probables

¿Qué evita que vea el carrito ajeno? AccesoCliente compara identidades; si no coinciden, responde **403** (salvo administrador).

¿El carrito puede cobrar o agendar? No. Acumula servicios; Reservas gestiona horas, orden y pago simulado.

¿Cómo sabe el precio correcto? CatalogoFeign consulta el servicio; no se confía en un precio inventado por el cliente.

Ubica en el código: CarritoController.agregar() · CarritoServices.agregar() · validarTope() · AccesoCliente.

[Saber por si preguntan]

El carrito se persiste en **MySQL**. Se comunica por **OpenFeign** para comprobar la existencia del cliente y los datos del tratamiento. No hay una llamada directa Carrito → Reservas en el código adjunto.

07 Diapositiva 4 · Reservas

Analogía: una agenda que aparta un turno mientras pagas.

[LEER EN VOZ ALTA] · QUÉ DECIR

“Reservas transforma la intención de compra en citas reales. Comprueba que el paciente exista, valida el horario, revisa cruces y asigna recursos disponibles. Al realizar checkout crea una orden y aparta los horarios durante diez minutos con estado PENDIENTE_PAGO. Si el pago de demostración se acepta, las citas pasan a PROGRAMADA; si vence el plazo, se liberan. También permite consultar, cancelar y reprogramar según las reglas del sistema.”

Tres ideas que debes dominar

- **Límites:** hasta **4 sesiones por compra**, **4 citas activas**, **1 sesión diaria** y **30 días** de anticipación.
- **Estado: PENDIENTE_PAGO → PROGRAMADA** (demo) o **EXPIRADA** (vencida).
- **Tiempo:** el bloqueo dura **10 min**; @Scheduled comprueba expiraciones cada **30 s**.

Tres preguntas probables

¿Cómo evita reservar una hora ocupada? Comprueba horarios y cruces existentes. Para alta concurrencia real añadiría bloqueos de BD.

¿El pago se cobra a un banco o a Yape? No. Es validación simulada de datos, adecuada para la demostración.

¿Quién puede crear una cita presencial sin pagar? La ruta **POST /reservas/registrar** exige rol **ADMINISTRADOR**.

Ubica en el código: ReservaController.checkout() · **ReservaServices.checkout()** · **pagar()** · **liberarRetencionesVencidas()**.

[Saber por si preguntan]

Hay autorización doble: **SecurityConfig** restringe acciones administrativas y **AccesoCliente** impide consultar/modificar citas ajenas.

08 Diapositiva 5 · Eureka

Analogía: el directorio del mall, no la puerta de seguridad.

[LEER EN VOZ ALTA] · QUÉ DECIR

“Eureka es el directorio de microservicios. Al iniciarse, cada servicio se registra con un nombre y ubicación. El Gateway no tiene que memorizar las direcciones físicas: pregunta al registro y envía solicitudes al servicio disponible. Nuestros clientes Feign hacen algo parecido para comunicarse entre servicios. Eureka facilita el descubrimiento y la escalabilidad, pero no valida contraseñas ni cifra conexiones; esa seguridad se trata en otras capas.”

Tres ideas que debes dominar

- **Servidor: @EnableEurekaServer**, puerto **8761**, registro y descubrimiento.
- **Clientes:** Gateway, Auth, Carrito y Reservas se anuncian y consultan el registro.
- **Balanceo:** con varias instancias, **Spring Cloud LoadBalancer** puede repartir llamadas entre ellas.

Tres preguntas probables

¿Eureka envía todos los mensajes? No. Da ubicaciones; Gateway/Feign envían las peticiones al destino.

¿Qué pasa cuando cae un servicio? Se pierde disponibilidad hasta que aparezca una instancia sana; el Gateway podría devolver **503**.

¿Eureka protege la red interna? No. Es descubrimiento; requiere protección propia y, en producción, TLS y controles de red.

Ubica en el código: ServidorEurekaApplication · Servidor-Eureka/application.yml · Gateway/application.yml (lb://).

[Saber por si preguntan]

En **docker-compose.yml** solo el Gateway publica el puerto de entrada de API; Eureka expone **8761** para la demo. Los servicios se registran por nombre, sin URL fija en cada cliente Feign.

09 CHEAT SHEET · una página

Definiciones de diez palabras o menos. Repásalas antes de exponer.

JWT
Token firmado con identidad, atributos y vencimiento.

Bearer
Credencial enviada dentro de Authorization.

HS256
Firma HMAC-SHA256 usando un secreto compartido.

Claim
Dato declarado dentro del JWT.

BCrypt
Hash lento para guardar y comprobar contraseñas.

Autenticación
Comprueba quién es el usuario.

Autorización
Decide qué puede hacer un usuario.

401 Unauthorized
Credencial ausente, incorrecta o expirada.

403 Forbidden
Identidad válida, pero acceso denegado.

Gateway
Puerta de entrada y enrutador de solicitudes.

Eureka
Registro para localizar instancias de servicios.

Service discovery
Descubrimiento dinámico de direcciones de servicios.

Load balancing
Distribución de solicitudes entre instancias disponibles.

OpenFeign
Cliente declarativo para llamar otros servicios HTTP.

OAuth2 Resource Server
Componente que valida tokens para proteger recursos.

Spring Security
Framework de autenticación y autorización en Spring.

HTTPS / TLS
Cifrado y autenticación del canal de comunicación.

CORS
Reglas del navegador sobre orígenes permitidos.

Stateless
Servidor sin sesión de login almacenada.

Checkout
Convierte sesiones elegidas en una orden temporal.

Frase de cierre: “Eureka encuentra; Gateway filtra; Auth identifica; Carrito reúne; Reservas confirma.”